

Тема урока: декоратор @property

1. Декоратор @property

Аннотация. Урок посвящен декоратору @property.

Декоратор @property

В предыдущем курсе мы познакомились с декораторами, однако до этого момента ни разу ими полноценно не пользовались.



Декоратор — функция, которая принимает в качестве аргумента функцию и возвращает новую функцию, которая расширяет функционал переданной.

Когда функция `property()` была добавлена в Python 2.2, синтаксис применения декораторов с помощью символа `@` еще не был доступен. Единственным способом создания свойств был вызов функции `property()` с соответствующим набором методов (геттер, сеттер и делитер). Синтаксис применения декораторов был добавлен в Python 2.4, и с того времени функция `property()` зачастую применяется именно как декоратор.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name                                # имя кошки

    @property
    def name(self):                                       # геттер свойства name
        return self._name

    @name.setter
    def name(self, name):                                # сеттер свойства name
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    @name.deleter
    def name(self):                                       # делитер свойства name
        del self._name
```

определяет в классе `Cat` свойство `name`, используя функцию `property()` в качестве декоратора.

Данный способ определения свойства заметно отличается от того, которым мы пользовались ранее. Нам больше не нужно определять три метода с разными именами `get_name`, `set_name` и `del_name`, вместо этого мы сразу определяем три метода с общим именем `name`, которое и является именем создаваемого свойства.

Сперва мы определяем метод `name()`, реализующий логику геттера, и декорируем его функцией `property()`. Другими словами, мы создаем свойство `name`, которое пока что доступно только для чтения. Далее мы снова определяем метод `name()`, реализующий логику сеттера, однако на этот раз декорируем его не функцией `property()`, а функцией `name.setter()`.

Дело в том, что все объекты `property` имеют три метода `getter()`, `setter()` и `deleter()`. Все они работают похожим образом. Например, метод `setter()` принимает в качестве аргумента некоторый метод `method` и возвращает новый объект `property`, который в качестве геттера и делитера имеет методы текущего объекта `property`, а в качестве сеттера — переданный метод `method`.

Когда мы декорируем второй метод `name()` функцией `name.setter()`, мы создаем новый объект `property` и переприсваиваем его имени `name`. Этот новый объект в качестве геттера имеет геттер предыдущего объекта `property`, а в качестве сеттера — метод, указанный при его создании.

Аналогично для свойства определяется и делитер. Мы создаем метод с тем же именем `name`, реализующий логику делитера, и декорируем его функцией `name.deleter()`, которая возвращает новый объект `property`, содержащий все методы предыдущего объекта `property` а также делитер, указанный при его создании. Этот новый объект присваивается имени `name` и становится окончательным свойством с полным набором методов.

Таким образом, применение функции `property()` в качестве декоратора — это последовательное создание новых свойств и переприсваивание их **одной и той же** переменной. Для наглядности мы можем записать данный процесс без использования синтаксиса декораторов.

Приведенный ниже код:

```

class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    name = property(get_name)          # свойство, имеющее только геттер

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = name.setter(set_name)       # свойство, имеющее геттер предыдущего
свойства, а также сеттер

    def del_name(self):
        del self._name

    name = name.deleter(del_name)     # свойство, имеющее геттер и сеттер
предыдущего свойства, а также делитер

```

эквивалентен коду, приведенному выше, и так же определяет свойство `name` с соответствующим набором методов (геттер, сеттер, делитер).

Примечания

Примечание 1. Использование функции `property()` в качестве декоратора позволяет достаточно просто создавать свойства, доступные только для чтения.

Приведенный ниже код:

```

class Cat:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

```

создает свойство `name`, доступное только для чтения.

Примечание 2. При создании свойства с помощью синтаксиса декораторов методы, реализующие для него логику геттера, сеттера и делитера, должны иметь одно и то же имя — имя создаваемого свойства. Если мы поступим иначе, например, дадим сеттеру и делитеру другие имена, мы получим не одно свойство, а три.

Приведенный ниже код:

```

class Cat:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):                # первое свойство name, доступное только для
чтения
        return self._name

    @name.setter
    def set_name(self, name):     # второе свойство set_name, доступное для
чтения и записи
        self._name = name

    @name.deleter
    def del_name(self):           # третье свойство del_name, доступное для
чтения и удаления
        del self._name

cat = Cat('Кемаль')

print(cat.name)
print(cat.set_name)
print(cat.del_name)

```

ВЫВОДИТ:

```

Кемаль
Кемаль
Кемаль

```

Примечание 3. Строку документации свойства, определяемого декоратором `@property`, необходимо указывать в геттере.

Приведенный ниже код:

```

class Cat:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        """Имя кошки. Доступно только для чтения."""
        return self._name

    @name.deleter
    def name(self):
        del self._name

cat = Cat('Кемаль')

```



